

Introducción a JavaScript

TUP Micaela Alancay

Introducción a JavaScript

- **JavaScript:** Es un lenguaje de **programación de alto nivel, dinámicamente tipado** y orientado a objetos.
 - **Tipado dinámico:** No exige definir el tipo de dato al crear una variable y permite pasarle cualquier valor (números, texto, booleanos) a las funciones.
- **Uso de JavaScript**
 - **Navegadores:** Añade interactividad y lógica a las páginas web. .
 - **Servidores:** Ejecuta código del lado del servidor mediante Node.js.
 - **Aplicaciones móviles:** Framework como React Native permiten el desarrollo de aplicaciones móviles usando JS

Creación del primer archivo .js y Entorno de Ejecución

- **Extensión de archivo:** El código fuente de JavaScript debe guardarse siempre en archivos con la extensión .js (ejemplo: app.js, index.js o script.js)
- **Nombres convencionales:** Se recomienda utilizar minúsculas, sin espacios ni caracteres especiales (usar camelCase o guiones medios: mi-script.js).
- **Formas de ejecución:**
 - En el Navegador: Enlazando el archivo en un documento HTML mediante la etiqueta **<script src="script.js"></script>**
 - En la Terminal (Node.js):Ejecutando el comando **node nombre-del-archivo.js**.

Sintaxis Básica y Comentarios

Los comentarios no alteran la memoria ni el flujo de ejecución del programa. Sin embargo, en el desarrollo profesional facilitan la mantenibilidad del software

1. `//` comentario en una sola línea
2. `/*` Comentario en multilinea `*/`

```
const PRECIO_BASE = 100; //Definición del precio unitario
```

Mecanismos de Declaración y Memoria

var

Ámbito de función (function scope) o global. Permite re-declaración y reasignación.

let

Ámbito de bloque (block scope {}). Permite reasignación pero **no** re-declaración en el mismo ámbito.

const

Ámbito de bloque. Inmutable en su referencia (no se puede reasignar ni re-declarar) y exige inicialización obligatoria.

Criterio	var	let	const
Ámbito (Scope)	Función / Global	Bloque ({})	Bloque ({})
Re-declaración	Permitida en el mismo ámbito	Error de Sintaxis (SyntaxError)	Error de Sintaxis (SyntaxError)
Reasignación	Permitida	Permitida	No permitida (TypeError)
Inicialización	Opcional	Opcional	Obligatoria

Comprobación Práctica de Ámbitos (Scope)

Ejemplo

```
// --- 1. COMPORTAMIENTO DE VAR ---  
✓ if (true) {  
  var x = 10;  
}  
console.log(x); // Imprime: 10 (Fuga de ámbito fuera del bloque 'if')
```

Comprobación Práctica de Ámbitos (Scope)

Ejemplo

```
// --- 2. COMPORTAMIENTO DE LET ---  
if (true) {  
  let a = 10;  
  console.log(a); // Imprime: 10 (Accesible dentro del bloque)  
}  
// console.log(a); // Uncaught ReferenceError: a is not defined  
  
// Reasignación y re-declaración con let:  
let c = 20;  
c = 30; // VÁLIDO: Reasignación de valor  
// let c = 40; // ERROR: SyntaxError: Identifier 'c' has already been declared
```

Comprobación Práctica de Ámbitos (Scope)

Ejemplo

```
// --- 3. COMPORTAMIENTO DE CONST ---  
if (true) {  
  const d = 10;  
  console.log(d); // Imprime: 10  
}  
// console.log(d); // Uncaught ReferenceError: d is not defined  
const f = 20;  
// f = 30; // ERROR: TypeError: Assignment to constant variable.
```

ReferenceError → variable no encontrar

TypeError → operación no permitida sobre un tipo/referencia

Criterios de Selección: ¿Cuándo usar var, let o const?

1. **Regla por defecto:** Declarar todas las variables con const.
2. **Si el valor debe cambiar:** Cambiar la declaración a let.
3. **Evitar totalmente var:** Se considera una práctica obsoleta en el desarrollo web

```
// 1. Usar const para valores fijos o referencias que no cambian
const PI = 3.14159;

// 2. Usar let solo cuando el valor deba reasignarse explícitamente
let acumulador = 0;
acumulador += 15;

// 3. NUNCA usar var en código ES6+ moderno
```

Sistemas de Tipos: Cadenas de Texto y Valores Numéricos

- **String:** Cadenas de caracteres encerradas entre comillas simples ('...'), dobles ("...") o plantilla de cadena (`...`).
- **Number:** Representa tanto números enteros (integers) como de punto flotante (floats). Implementa el estándar IEEE 754 de 64 bits.
- Uso de `console.log()` para salida por consola estándar en cualquier entorno.

```
// Cadena de texto (String)
const nombre = "Pepito";
console.log(nombre); // Imprime: Pepito
// Números (Number): Enteros y Decimales
const edad = 26;
const iva = 21; // Entero
const total = 1210.22; // Decimal (Float)
console.log(`Edad: ${edad}, IVA: ${iva}%, Total: ${total}`);
```

Template Literals (Plantillas de Cadena)

- **¿Qué son?:** Una forma moderna y cómoda de armar textos (cadenas) en JavaScript
- **Sintaxis:** Se delimitan con comillas invertidas (``) en lugar de simples (') o dobles (").
- **Interpolación (\${ }):** Permite meter variables o cálculos dentro del texto sin usar el signo +.
- **Multilínea:** Podés hacer saltos de línea apretando Enter, sin usar código raro como `\n`.

```
const usuario = "Micaela";
const materia = "Introducción a Desarrollo Web";

// Concatenación tradicional (Obsoleta)
const mensaje1 = "Hola " + usuario + ", bienvenida a " + materia + ".";

// Template Literals (Moderna y recomendada)
const mensaje2 = `Hola ${usuario}, bienvenida a ${materia}.`;
```

Booleanos, Ausencia de Valor y Colecciones Secuenciales

- **Boolean:** Representa los valores lógicos de verdad: true o false.
- **Undefined:** Estado por defecto de variables declaradas a las que no se les ha asignado un valor explícito.
- **null:** Ausencia *intencional* y explícita de valor asignada por el desarrollador
- **Array (Arreglo):** Estructura de datos que almacena elementos ordenados de forma secuencial mediante índices numéricos basados en cero (0).

```
// Booleano
const esEstudiante = true;
// Undefined (Declarada pero no asignada)
let notaFinal;
console.log(notaFinal); // Imprime: undefined
// Array básico
const materias = ["PAW", "Base de Datos", "Programación I"];
console.log(materias[0]); // Imprime: PAW (Primer elemento, índice 0)
```

Operaciones Aritméticas e Incrementales

1. Operador de asignación simple (=)
2. Operadores combinados de asignación y aritmética (+=, -=, *=, /=, %=).
3. Módulo o resto (%): Retorna el residuo entero de una división.

Operador	Ej. Simplificado	Equiv. Completa
Suma y Asignación	<code>x += 4;</code>	<code>x = x + 4;</code>
Resta y Asignación	<code>x -= 4;</code>	<code>x = x - 4;</code>
Multiplicación y Asignación	<code>x *= 4;</code>	<code>x = x * 4;</code>
División y Asignación	<code>x /= 4;</code>	<code>x = x / 4;</code>
Módulo y Asignación	<code>x %= 4;</code>	<code>x = x % 4;</code>

Operadores de Comparación e Igualdad Estricta

- `==` (Igualdad abstracta): Compara valores realizando **coerción implícita de tipos**.
- `!=` (Desigualdad abstracta): Evalúa si los valores son distintos tras convertir tipos.
- `===` (Igualdad estricta): Compara **valor y tipo de dato** sin realizar conversiones.
- `!==` (Desigualdad estricta): Evalúa si el valor o el tipo son distintos.

```
console.log(5 == "5");    // true
console.log(5 === "5");   // false
console.log(0 == false);  // true
console.log(0 === false); // false
console.log(5 !== "5");   // true
```

Operador Ternario: Evaluación Condicional en Una Línea

- Expresión concisa para evaluar una condición booleana.
- **Sintaxis:** condición ? valorSiVerdadero : valorSiFalso;
- A diferencia del if, el operador ternario es una expresión que retorna un valor directamente.

```
let edad = 25;  
// Uso del operador ternario para asignar un valor condicional  
let mensaje = edad >= 18 ? "Mayor de edad" : "Menor de edad";  
console.log(mensaje); // Imprime: Mayor de edad
```

Al ser una expresión, el operador ternario puede asignarse directamente a una variable i usarse dentro de un retorno de función. Debe reservarse para condiciones simples de una sola línea a fin de no perjudicar la legibilidad del código

Evaluación de Veracidad: Valores Falsy y Truthy

- En contextos booleanos (como en la condición de un if), cualquier valor se evalúa como verdadero (truthy) o falso (falsy).
- Existen exactamente **8 valores falsy** en JavaScript.
- Todo lo demás es evaluado como **truthy** (incluyendo arrays y objetos vacíos [] y {}).

Lista de Valores Falsy: false, 0 (cero entero), -0 (cero negativo), 0n (BigInt 0), "" (cadena vacía), null, undefined y NaN (Not a Number)

```
let nombre = ""; // Cadena vacía -> Falsy
if (nombre) {
  console.log(`Usuario: ${nombre}`);
} else {
  console.log("El nombre está vacío."); // Se ejecuta este bloque
}
```


Estructura Condicional if - else if - else

Las estructuras **if / else if / else** evalúan condiciones secuencialmente de arriba hacia abajo. En el momento en que una condición resulta verdadera (true), su bloque asociado se ejecuta y el motor ignora las ramas restantes.

```
const edad = 20;
if (edad >= 25) {
    console.log("Es un adulto.");
} else if (edad >= 18) {
    console.log("Es un adulto joven."); // Se ejecuta este bloque
} else {
    console.log("Es menor de edad.");
}
```

Bucle for: Iteración Contada por Índices

- Bucle determinado para ejecutar un bloque de código un número conocido de veces.
- Consta de 3 partes: **inicialización**, **condición** de permanencia e **incremento**.

```
const limitePasos = 5;
for (let paso = 1; paso <= limitePasos; paso++) {
  console.log(`Estoy dando el paso número: ${paso}`);
}
```

Bucle while: Iteración Basada en Condición

- Repite un bloque de código mientras la condición evaluada continúe siendo true.
- **Atención:** La variable de control debe reasignarse dentro del bucle para evitar bucles infinitos.

```
let contador = 0; // Se declara con 'let' para permitir su incremento
while (contador < 3) {
    contador++; // Reasignación indispensable
    console.log(`Contador es igual a: ${contador}`);
}
```

Estructura switch: Múltiples Casos de Evaluación Directa

- Estructura limpia para evaluar múltiples opciones sobre un mismo valor de entrada.
- **Uso de break:** Indispensable para salir de la estructura y evitar la ejecución en cascada (fall-through).
- **default:** Opción de resguardo si ningún caso coincide.

```
const tipoFruta = "Manzanas";
switch (tipoFruta) {
  case "Naranjas":
    console.log("Las Naranjas cuestan $5");
    break;
  case "Manzanas":
    console.log("Las Manzanas cuestan $10");
    break;
  case "Fresas":
    console.log("Las Fresas cuestan $15");
    break;
  default:
    console.log(`Disculpa, no comercializamos: ${tipoFruta}`);
    break;
}
```

Introducción a Funciones

- Una **función** es un bloque de código organizado para realizar una tarea específica.
- Recibe datos opcionales de entrada (**parámetros**) y puede devolver un resultado (**retorno**).
- **Retorno por defecto:** Si no incluye la instrucción **return**, la función devuelve **undefined** automáticamente.

```
1 // 1. Función con retorno explícito
2 function obtenerSaludo(nombre) {
3   return `Hola, ${nombre}!`;
4 }
5 const saludo = obtenerSaludo("Micaela");
6 console.log(saludo); // Imprime: "Hola, Micaela!"
7
8 // 2. Función sin retorno (devuelve undefined por defecto)
9 function mostrarMensaje() {
10   console.log("Ejecutando la función...");
11   // No hay instrucción 'return'
12 }
13 const resultado = mostrarMensaje();
14 console.log(resultado); // Imprime: undefined
```

Funciones Declarativas

- **Funciones Declarativas:** Se definen utilizando la palabra clave `function` seguida del nombre de la función
- **Parámetros por defecto:** Permiten establecer un valor inicial para un parámetro en caso de que no se le pase ningún valor al llamar a la función.

```
// 1. Declaración e invocación básica
function sumar(a, b) {
  return a + b;
}
console.log(sumar(5, 3));
```

Funciones Flecha (Arrow Functions)

- **Son anónimas:** No llevan nombre propio después de la declaración, por lo que **siempre se asignan a una variable** (**const**).
- **Sintaxis moderna (=>):** Sustituyen la palabra clave **function** por una flecha (**=>**), logrando un código más corto.
- **Retorno implícito:** Si el cuerpo de la función ocupa una sola línea, se pueden omitir las llaves **{ }** y la palabra **return**.

```
// 0 parámetros: Paréntesis obligatorios
const saludar = () => "¡Hola mundo!";
// 1 parámetro: Paréntesis opcionales
const incrementar = e => e + 1;
// 2 parámetros: Paréntesis obligatorios
const sumar = (a, b) => a + b;
```

EJERCICIOS

1. **dividir (Declarativa + Validador):** Crear una función declarada `dividir(a, b)` que devuelva la división, pero verifique con un `if` que `b` no sea cero.
2. **saludar (Declarativa + Template Literals):** Crear una función declarada `saludar(nombre = "Invitado")` que use **parámetros por defecto** y devuelva el mensaje usando **template literals** (``Hola, ${nombre}``).
3. **calcularArea (Función Flecha - Retorno Implícito):** Crear una **función flecha** en una sola línea `calcularArea(largo, ancho)` para calcular el área de un rectángulo.
4. **esPar (Función Flecha):** Crear una **función flecha** `esPar(numero)` que devuelva `true` si es par y `false` si es impar.
5. **encontrarMayor (Flecha u Operador Ternario):** Crear una función `encontrarMayor(a, b)` que retorne el número más grande utilizando el **operador ternario**.